



Vital-ERP — Issues Report

All 19 findings from a full code review — explained in detail

Vital Security PLC Payroll & HR Management System

Backend: Express + TypeScript + MongoDB • Frontend: React 18 + Vite

Found by Cline (AI code review) • 2026-09-14 • Both typechecks pass (0 errors) — these are **logic** bugs,
not compile errors



How to read this report

Every issue is written the same way:

- **WHERE** — the exact file and line numbers, so a developer can find it in seconds
- **WHAT'S WRONG** — the problem explained in simple words
- **THE FAILING LOGIC / MATH** — shown step by step, where it applies
- **REAL-LIFE EFFECT** — what actually goes wrong for the company
- **SUGGESTED FIX** — how a developer would repair it

Severity colors: ● **PAYROLL-CRITICAL** touches real money • ● **FUNCTIONAL** features misbehave
• ● **SECURITY** exploitable weaknesses • ● **HOUSEKEEPING** hygiene, environment, readiness

Note: the TypeScript compiler passes with 0 errors on both backend and frontend — none of these are syntax problems. They are all **logic and functional** bugs that compile fine but do the wrong thing.

Summary table (all 19 at a glance)

#	Severity	One-line description	Main file
1	● Payroll	Journal entry unbalanced by employer pension → accounting silently never recorded	payrollJournal.service.ts
2	● Payroll	Guard pension calculated on full 720h month, not actual worked hours → net pay can go negative	payrollCalculation.service.ts:156
3	● Payroll	Staff overtime is calculated but never added to gross or net pay	payrollCalculation.service.ts:203-256
4	● Payroll	Changing a guard's pay rate in the app has no effect on payroll	guard.service.ts / payrollCalculation.service.ts:74
5	● Payroll	Guard payroll needs period OPEN, staff payroll needs LOCKED — same field, contradictory rules	guardPayroll.service.ts:44
6	● Functional	Holiday-OT multiplier hardcoded to 2.5, ignores the configurable setting	payrollCalculation.service.ts:152
7	● Functional	Manually entered staff overtime gets silently wiped by recalculation	officePayroll.service.ts:39
8	● Functional	Loans deducted every month forever — no balance tracking or auto-finish	payrollCalculation.service.ts:314,423
9	● Functional	Guards without a current site assignment get no payroll record at all	payrollCalculation.service.ts:281
10	● Functional	Attendance on the last day of the month is excluded from pay (date boundary)	payrollCalculation.service.ts:297
11	● Functional	48-hour relief rule blocks the reliever from clocking in → sites stand empty (needs business confirmation)	attendance.service.ts:32-45
12	● Security	JWT secret falls back to a well-known string → login tokens can be forged	config/env.ts:7
13	● Security	Uploaded employee documents (IDs, contracts) downloadable without logging in	app.ts:50
14	● Security	All 8 demo accounts share one password that is committed in the source code	seed.ts:26

15	● Security	CORS accepts requests from any website	app.ts:45
16	● Security	No rate limiting on login — passwords can be brute-forced forever	app.ts
17	● Housekeeping	MongoDB is not installed on this computer — nothing can run yet	environment
18	● Housekeeping	Code hygiene: dead duplicate modules, duplicate DB indexes, junk files in repo, empty README, stale tsc-check	modules/, models/, repo root
19	● Housekeeping	Production readiness: three different "month" constants hardcoded, no automated tests, frontend API path needs a reverse proxy	various

PART 1 — Payroll-critical issues (these touch real money)

ISSUE 1 The employer's pension breaks every accounting entry — and the failure is silent

WHERE

`backend/src/modules/finance-accounting/payrollJournal.service.ts` lines 17–104 (guard) and 106–180 (staff); balance check in `journal.service.ts` line 32; the swallowing `try/catch` in `guardPayroll.service.ts` line 277 and `officePayroll.service.ts` line 261.

WHAT'S WRONG

When Finance marks a salary as PAID, the system writes a double-entry accounting record (journal entry). Double-entry bookkeeping has one golden rule: **total debits must equal total credits** — every birr must come from somewhere and go to somewhere. The entry correctly debits (adds expense for) the salary, and also debits the **employer's 11% pension contribution** as a company expense. But it **never records the credit side** of that 11% — the money the company now owes to the pension agency.

THE FAILING MATH

```
Debits   = grossPay + employerPension
Credits  = netPay + incomeTax + employeePension + loanDeduction
          = (grossPay - incomeTax - employeePension - loan)
            + incomeTax + employeePension + loan
          = grossPay
```

```
Difference = employerPension ≠ 0 → journal.service.ts line 32 throws:
"Debit total (...) must equal credit total (...)"
```

Bonus bug: the entry builder also tries to split overtime into its own expense line using `record.otPay` — but the calculation engine never fills that field (it fills `regularOtPay` instead). That branch is dead code, and if it ever ran, its index-based `lines.splice()` math would misplace lines.

REAL-LIFE EFFECT

The balance check throws an error **every single time a salary is paid**. The workflow service catches this error and only prints it to the console (`console.error('[GuardPayroll] Failed to post journal entry')`), so:

- ✓ The payment itself succeeds — status becomes PAID, Finance sees everything as normal
- ✗ **No accounting record is ever created — for any salary, ever**
- ✗ The Finance "Control Tower", journal list, and dashboard summary slowly become empty/wrong, and nobody gets an error message

SUGGESTED FIX

Add the missing credit line to both guard and staff postings: account `2210 Pension Payable`, credit = `employerPension` (or a separate `Employer Pension Payable` account, e.g. 2215, to keep the employee's 7% and the company's 11% distinct). Also replace `record.otPay` with `record.regularOtPay` and rewrite the line-insertion logic so it doesn't depend on fragile indexes.

● ISSUE 2

Guards' pension is calculated on a full month they never worked — net pay can go NEGATIVE

📍 WHERE

`backend/src/modules/hr/finance/payrollCalculation.service.ts` lines 149–161 (`calculateGuardPayroll`); the 720-hour constant at line 16 (`GUARD_MONTHLY_HOURS = 720`).

⚠️ WHAT'S WRONG

The engine computes two different numbers and then mixes them up:

```
normalSalary = 720 hours × hourly rate      ← the FULL standard month
grossPay      = workedSalary + OT + holiday  ← what the guard ACTUALLY earned
pension       = calculatePension(normalSalary) ← ⚠️ pension on the FULL month!
```

The pension — both the 7% deducted from the guard AND the 11% the company pays — is always calculated on the full 720-hour standard salary, even if the guard only worked a fraction of the month. The `pensionTaxBase` setting ("NORMAL_SALARY_ONLY" vs "GROSS_PAY") is applied afterwards to pick `baseComponent`, but the pension itself was already computed on the full `normalSalary`.

🧮 THE FAILING MATH (WORKED EXAMPLE)

```
Guard hourly rate = 15 birr. Worked only 200 normal hours this month.
grossPay          = 200 × 15                      = 3,000 birr
normalSalary      = 720 × 15                      = 10,800 birr
employeePension   = 10,800 × 7%                   = 756 birr (should be ~210)
incomeTax         = on 3,000 (gross)               = ~200 birr
netPay            = 3,000 - 756 - 200 - 0           = 2,044 birr (over-deducted 546!)
```

Extreme case: worked 50 hours (750 birr gross) → pension 756 → netPay = -6 birr.
A guard would OWE the company money for working.

★ REAL-LIFE EFFECT

- Every guard who works less than the standard month is **over-deducted** pension
- The **company over-pays 11% employer pension** on hours never worked — this is almost certainly the "problem on the employer part" the original developer sensed
- In extreme under-work cases net pay goes **negative** — the record will show the company paying a guard a negative salary

🔧 SUGGESTED FIX

Compute pension from the correct base: when `pensionTaxBase === 'NORMAL_SALARY_ONLY'`, use the **worked** basic (e.g., `workedSalary`, or gross if that's the business definition — needs one business decision), not

`standardMonthlyHours × rate` . Also decide and document whether guards' tax should be on grossPay (it currently is, unlike staff who exclude non-taxable allowances — see Issue 19).

● ISSUE 3 Staff overtime is carefully calculated... and then never paid

📍 WHERE

`payrollCalculation.service.ts` , `calculateStaffPayroll` : gross summing at lines 203–207, OT computing at lines 252–256, net pay at line 264.

⚠️ WHAT'S WRONG

The order of operations is wrong. The function:

1. **First** loops over the salary-structure earnings and sums `grossSalary` (lines 203–207). If an "OT" earning component exists, it reads `record.overtime` — which is still **0** for a fresh record.
2. **Then** (lines 252–256) it converts OT *hours* into OT *pay* and sets `record.overtime = regularOtPay + holidayOtPay` — **after** gross was already summed.
3. **Finally** `netPay = grossSalary - totalDeductions` (line 264) — using the gross that never included the OT.

📊 THE FAILING MATH

Staff wage 6,000, worked 10 regular OT hours (hourly = 6,000/192 = 31.25).

<code>regularOtPay</code>	<code>= 10 × 31.25 × 1.5 = 468.75</code>	← computed, saved to <code>record.overtime</code>
<code>grossSalary</code>	<code>= 6,000 (+ allowances)</code>	← OT NOT included (summed too early)
<code>netPay</code>	<code>= grossSalary - deductions</code>	← OT missing here too

Result: the record shows overtime = 468.75 on screen, but the employee is PAID 0 of it.

★ REAL-LIFE EFFECT

Office staff who work overtime see the OT amount stored on their payslip record but **receive none of it**. Tax is also never applied to it (it never entered `taxableSalary`). And when the (already broken — Issue 1) journal entry fires, OT is missing there too.

🔧 SUGGESTED FIX

Compute OT pay **before** summing gross, then include it in `grossSalary` and `taxableSalary` before the tax/pension/net steps.

● ISSUE 4

Changing a guard's pay rate in the app does nothing to their salary

📍 WHERE

`guard.service.ts` `updateGuardPayRate` (lines 145–156) saves `assignment.hourlyRate` ; `registerGuard` (line 54) saves `GuardProfile.rate` ; but `payrollCalculation.service.ts` `resolveGuardRates` (lines 74–139) never reads either field.

⚠️ WHAT'S WRONG

The app has TWO places where a guard's hourly rate can be set: on registration (`GuardProfile.rate`) and per site-assignment (`PrimarySiteAssignment.hourlyRate` , editable from the Guards screen). Both save correctly. But when payroll resolves each guard's rate, its priority chain is:

1. `Contract.wage` (if greater than 0)
2. `SalaryStructure BASIC defaultRate` (divided by 720)
3. A random `PayrollRate` document (fallback)

... `assignment.hourlyRate` and `GuardProfile.rate` are NEVER consulted.
The function even receives the assignment as a parameter – and ignores it.

💥 REAL-LIFE EFFECT

An Operations manager raises a guard's hourly rate (promotion, better-paying site) → the next payroll silently calculates with the **old** rate. No error, no warning — money paid at the wrong rate.

🔧 SUGGESTED FIX

Decide the intended priority (business decision), then make `resolveGuardRates` actually read `assignment.hourlyRate` (likely between contract wage and structure default). If the assignment rate is meant to be display-only, remove the edit feature — but "editable field that is ignored" is the worst of both worlds.

● ISSUE 5

Guard payroll and staff payroll fight over the same "month switch"

📍 WHERE

Guard rule: `guardPayroll.service.ts` line 44 (throws unless period is OPEN). Staff rule: `payrollCalculation.service.ts` line 354 (throws unless LOCKED). Official state machine: `payrollPeriod.service.ts` lines 46-50 (DRAFT, OPEN, CLOSED, LOCKED chain). The bypasses: `staffAttendance.service.ts` `lockPeriod` (line 185, jumps straight to LOCKED) and `unlockPeriod` (line 210, back to OPEN).

⚠️ WHAT'S WRONG

Both payroll modules use the **same** `PayrollPeriod.status` field — one status shared by two different workflows — with contradictory expectations:

Action	Required period status
Generate guard payroll	must be OPEN
Generate staff payroll	must be LOCKED
HR locks month (staff attendance)	sets status = LOCKED (skipping CLOSED)
HR unlocks month	sets status = OPEN (the official chain only allows this from DRAFT)

There are effectively **two competing state machines** writing to one field: the official transition map in `payrollPeriod.service.ts`, and direct jumps in `staffAttendance.service.ts`.

✳️ REAL-LIFE EFFECT

The moment HR locks the month so staff payroll can be generated, **guard payroll generation breaks** with "Period must be OPEN". The two payrolls cannot follow their intended order in the same month. Unlocking to fix it re-opens the staff period too, letting anyone edit staff attendance that was supposed to be frozen.

🔧 SUGGESTED FIX

Separate the concerns: either (a) add independent flags — e.g., `guardAttendanceLocked` and `staffAttendanceLocked` — while keeping one shared period lifecycle, or (b) split into two period models. Option (a) is the small surgical fix.

PART 2 — Functional issues (features that misbehave)

ISSUE 6 The holiday-overtime multiplier is hardcoded, ignoring the configurable setting

WHERE

`payrollCalculation.service.ts` line 152: `const holidayOtPay = record.holidayOtHours * record.normalRate * 2.5;`

WHAT'S WRONG

The salary structure has a configurable `holidayOtMultiplier` (default 2.5), and `resolveGuardRates` carefully loads it and even computes a `holidayOtRate`, which is stored on the record. But when the actual holiday-OT pay is computed, the code ignores all of that and multiplies by a **hardcoded 2.5**.

REAL-LIFE EFFECT

If the company changes the holiday-OT multiplier in Settings (say to 2.0), nothing changes in real pay — the setting is decorative. Pay is also computed from `normalRate x 2.5` rather than from the resolved `holidayOtRate`, so any future per-guard rate variation is ignored too.

SUGGESTED FIX

Use `record.holidayOtHours * record.holidayOtRate` (the resolved rate is already stored on the record).

● ISSUE 7

Manually entered staff overtime gets silently wiped by recalculation

📍 WHERE

`officePayroll.service.ts` `updateSalaryInputs` (lines 39-76) accepts a manual `overtime` amount;
`payrollCalculation.service.ts` line 256 overwrites `record.overtime = regularOtPay + holidayOtPay`.

⚠️ WHAT'S WRONG

HR can type an overtime **amount** directly into a staff payroll record. But the calculate step overwrites that field with its own computation from OT **hours** — which are 0 if HR only entered an amount. Depending on order of operations, the amount is either clobbered to 0 or counted in gross but displayed as 0 (see Issue 3 for the ordering bug).

💥 REAL-LIFE EFFECT

Numbers on the payslip screen do not match the math that was paid. HR inputs silently disappear. Auditors will find records where displayed overtime = 0 but net pay includes it (or vice versa).

🔧 SUGGESTED FIX

Remove the manual `overtime` amount field and let HR enter OT **hours** only (the engine already prices hours correctly), or make the calculation respect a manual override flag.

● ISSUE 8

Loans are deducted every month forever — no balance, no end

📍 WHERE

`payrollCalculation.service.ts` lines 314–319 (guards) and 423–425 (staff): `Loan.find({ employeeId, status: 'ACTIVE' })` then add `monthlyDeduction`.

⚠️ WHAT'S WRONG

Every payroll generation reads all loans with status ACTIVE and deducts their `monthlyDeduction`. There is no `remainingBalance`, no "months left" counter, no decrement after each payday, and no automatic completion. The only way a loan stops being deducted is if a human remembers to flip its status manually.

💥 REAL-LIFE EFFECT

A guard borrows 6,000 birr repayable over 6 months at 1,000/month. Unless somebody manually deactivates the loan after month 6, the system keeps deducting 1,000 birr **every month indefinitely** — an endless silent salary cut. This is both an employee-harm bug and a lawsuit waiting to happen.

🔧 SUGGESTED FIX

Add `totalAmount` / `remainingBalance` (or `monthsRemaining`) to the Loan model; at each payroll, deduct `min(monthlyDeduction, remainingBalance)` and decrement it; auto-set status to COMPLETED when it reaches zero.

● ISSUE 9

Guards without a current site assignment get NO payroll at all

📍 WHERE

`payrollCalculation.service.ts` line 281: `generateGuardPayrollRecords` starts with `PrimarySiteAssignment.find({ isCurrent: true })` and loops only over those.

⚠️ WHAT'S WRONG

Payroll generation iterates over **site assignments**, not over **guards**. A guard who exists as an employee but has no current site assignment (new hire not yet placed, or assignment accidentally ended) is simply never visited — no record, no error, no "skipped" list (unlike staff payroll, which does report skips).

💥 REAL-LIFE EFFECT

A working guard silently receives no salary for the month and — because nothing is reported — nobody notices until the guard complains. With ~227 guards, one missed assignment row equals one unpaid person.

🔧 SUGGESTED FIX

Generate from `Employee.find({ category: 'GUARD', status: 'ACTIVE' })` and treat missing site assignment as a flagged/zero-hour record (or at least add the guard to a "skipped" report like the staff flow already does).

● ISSUE 10

Attendance worked on the last day of the month is excluded from pay

📍 WHERE

`seed.ts` line 237 (and `staffAttendance.service.ts` line 246): `endDate: new Date(year, month + 1, 0)` — that is the **first second** (00:00) of the last day. Query: `payrollCalculation.service.ts` lines 295–298 use `date: { $gte: period.startDate, $lte: period.endDate }`.

⚠️ WHAT'S WRONG

A date constructed as `new Date(y, m, 0)` points to midnight at the START of the last day. Attendance records dated on the last day (with any time-of-day component) fall **after** `endDate`, so the `$lte` comparison excludes them.

💥 REAL-LIFE EFFECT

Every guard's hours worked on the final day of the month are silently dropped from that month's payroll (they may or may not resurface in the next period, depending on how dates get stored). If most guards work the last day, that is one full day of pay lost — every month, for everyone.

🔧 SUGGESTED FIX

Store the period end as the last millisecond of the last day (`new Date(y, m, 0, 23, 59, 59, 999)`) or query with `date: { $gte: startDate, $lt: firstDayOfNextMonth }`.

● ISSUE 11

The 48-hour "relief" rule blocks the reliever — sites could stand empty (needs business confirmation)

📍 WHERE

`attendance.service.ts` `clockIn`, lines 32–45 (`FORTY_EIGHT_HOURS_MS` check).

⚠️ WHAT'S WRONG

When a guard tries to clock in at a site where they were declared as a reliever, the system **blocks them if the previous guard clocked out less than 48 hours ago**:

```
"Predecessor clocked out less than 48 hours ago.  
Cannot clock in as relief. X hours remaining."
```

For a security company this looks **inverted**: normally the reliever must take over immediately so the post is never empty. A common anti-collusion rule would instead stop the **outgoing** guard from returning to the same site within 48h — but as coded, it blocks the **incoming** reliever.

💥 REAL-LIFE EFFECT

If taken literally, every handover forces the site to stand unguarded for up to 48 hours — the Coverage Dashboard would show red everywhere and client contracts (guards on post 24/7) would be violated. Alternatively, if this really is the company's rule, then the error message and the "no predecessor on duty" event logic contradict it.

🔧 SUGGESTED FIX

Ask the original developer / business which rule was intended. If the rule is "outgoing guard can't return within 48h", the query should check the predecessor's own guardId re-clocking-in, not the declared reliever's.

PART 3 — Security issues (exploitable weaknesses)

 **ISSUE 12** The JWT secret has a well-known fallback — login tickets can be forged

WHERE

`backend/src/config/env.ts` line 7: `jwtSecret: process.env.JWT_SECRET || 'fallback-secret-change-me'`

WHAT'S WRONG

Login sessions are "tickets" (JWT tokens) signed with a secret key. If the `.env` file is missing or missing the `JWT_SECRET` line, the app does not fail — it silently uses the string `'fallback-secret-change-me'`, which is committed to the repository for everyone to see.

REAL-LIFE EFFECT

Anyone who knows this string can generate valid tokens for **any user, any role — including SUPER_ADMIN** — without knowing any password. They could approve payroll, mark salaries paid, or read every employee's data. And because the failure is silent, the system would run insecurely with no hint.

SUGGESTED FIX

Fail fast: at startup, if `NODE_ENV === 'production'` and no `JWT_SECRET` is set, throw an error and exit. Also rotate the secret and use a long random value (e.g., 64+ random characters).

● ISSUE 13

Uploaded employee documents are downloadable without logging in

📍 WHERE

```
backend/src/app.ts line 50: app.use('/uploads', express.static(path.join(process.cwd(),  
'uploads')))
```

⚠️ WHAT'S WRONG

The `/uploads` folder holds employee documents (ID cards, contracts, certificates — uploaded via the protected `/api/files` endpoint, max 20 MB). But the folder itself is served as **static files with no authentication**. The upload endpoint checks login and permissions — yet the files it produces are then served to anyone.

💥 REAL-LIFE EFFECT

Anyone on the network (or the internet, if the server is exposed) who obtains or guesses a file URL — file names are `timestamp-randomNumber.ext`, which is weak protection — can download employees' private documents. No login, no log entry. An auditor would call this a data-leak path.

🔧 SUGGESTED FIX

Remove the static route; serve uploads through an authenticated endpoint that verifies the JWT, checks role permissions, and logs every download.

● ISSUE 14

One password for all accounts, committed in the source code

📍 WHERE

`backend/src/seed.ts` line 26: `bcrypt.hash('password123', 12)`, applied to all 8 demo accounts (lines 45–64).

⚠️ WHAT'S WRONG

Every demo account — Super Admin, System Admin, HR, Finance, Operations, Head, CEO, Guard — uses the same well-known password `password123`, and the seeding script (with the password in plain text) is public in the repository.

💥 REAL-LIFE EFFECT

Fine for a demo, dangerous for anything real: anyone who has seen this repo can log in as CEO or Finance. There is also no "must change password on first login" mechanism to force users off it.

🔧 SUGGESTED FIX

Before real use: randomize seed passwords, print them once, force a password change at first login, and remove the seed accounts entirely from production databases.

● ISSUE 15

CORS accepts requests from any website

📍 WHERE

`backend/src/app.ts` line 45: `app.use(cors())` with no options.

⚠️ WHAT'S WRONG

CORS decides which websites may call the API from a browser. With no configuration, **every** website is allowed.

💥 REAL-LIFE EFFECT

A malicious page visited by an employee (while their login token is in memory in another tab's local storage) can craft requests to the API with fewer browser-level restrictions. In practice this mostly matters together with the token's long 24h life and localStorage storage. Low effort to fix, standard hygiene.

🔧 SUGGESTED FIX

Restrict CORS to the real frontend origin(s): `cors({ origin: ['https://hr.yourcompany.com'] })`, configurable via `.env`.

● ISSUE 16

No rate limiting on the login endpoint

📍 WHERE

`app.ts` / `auth.routes.ts` — no `express-rate-limit` (or similar) middleware anywhere; the backend's dependency list confirms it is not installed.

⚠️ WHAT'S WRONG

Nothing limits how many login attempts one client can make per minute/hour. The login endpoint happily answers unlimited tries.

💥 REAL-LIFE EFFECT

A script can try thousands of password guesses per minute against `POST /api/auth/login` — "password spraying" all 8 known usernames with the world's most common passwords. Combined with Issue 14 (known usernames), the only protection is password strength.

🔧 SUGGESTED FIX

Add `express-rate-limit` globally (e.g., 300 req/15min) and a strict limiter on auth routes (e.g., 5 attempts/15min per IP), plus account lockout or exponential backoff after repeated failures.

● PART 4 — Housekeeping & readiness

● ISSUE 17

MongoDB is not installed on this computer — nothing can run yet

📍 WHERE

Environment check performed during this review: port 27017 refuses connections; `mongod` is not on PATH.

💥 REAL-LIFE EFFECT

The backend starts, tries to connect to `mongodb://127.0.0.1:27017/vitalpayroll`, fails, and exits. The system cannot be started, seeded, or tested on this machine until MongoDB Community Server is installed (as a Windows Service).

🔧 SUGGESTED FIX

Install MongoDB Community Server for Windows, keep the default port 27017, enable "Install as a Service", then run `npm run seed` in `backend/`.

Code hygiene: dead duplicate modules, duplicate indexes, junk in the repo

📍 WHERE

- **Dead duplicate modules:** `backend/src/modules/` contains BOTH the live `hr/` tree AND older copies of the same modules at the top level (`attendance/` , `finance/` , `guards/` , `guardPayroll/` , `officePayroll/` , `sites/` , `staffAttendance/` , `employees/`). Only the `hr/` copies are mounted in `app.ts` — the rest still compiles but is never used.
- **Duplicate indexes:** Mongoose warns on every startup about duplicate indexes on `email` , `employeeCode` , `siteCode` , `code` — schemas declare `unique: true` AND a manual `schema.index()` for the same field (e.g., `Employee.ts` line 80).
- **Junk files:** Word lock files (`~$tal_*.docx`) and test logs (`*_log.txt` , `backend_err.txt`) are committed; no `.gitignore` covers them.
- **Stale error file:** `tsc-check.txt` lists TypeScript errors that were already fixed (verified: current code passes cleanly) — misleading for anyone reading it.
- **Empty README:** `README.md` contains just "Vital-ERP / ERP".

💥 REAL-LIFE EFFECT

A developer can fix a bug in a dead duplicate module and see zero effect in the app — a classic time-waster. Repo clutter hides real signals. Startup warnings train people to ignore warnings.

🔧 SUGGESTED FIX

Delete the unused module copies; remove redundant `schema.index()` lines; add a `.gitignore` with `~$*` and log patterns and remove the junk; delete or regenerate `tsc-check.txt` ; write a real README (the setup section of the main documentation can be copied in).

path📌 **WHERE**

- **Three different "one month" constants hardcoded:** guards use `GUARD_MONTHLY_HOURS = 720` (payrollCalculation.service.ts line 16), staff OT uses `/192` (line 242), staff absence uses `/22` days (line 419). Three definitions of a month in one engine; a policy change requires editing code.
- **No automated tests:** only PowerShell smoke scripts in the repo root; no Jest/Vitest unit or integration tests. Payroll math is exactly the code where a wrong bracket silently mispays 227 people.
- **Dev-only API path:** the frontend calls `baseUrl: '/api'`, which only works because Vite proxies it in dev mode (vite.config.ts). In production a reverse proxy (nginx/IIS) or a configurable API URL is required.

💥 **REAL-LIFE EFFECT**

The system works in development but has no safety net (tests) and needs deployment configuration before serving real users. Any change to tax rules, hours, or working days is a code change instead of a settings change.

🔧 **SUGGESTED FIX**

Move the month constants into Settings/PayrollConfig (the tax and pension rules already are — follow that pattern); add Vitest/Jest unit tests for the calculation engine (tax brackets, pension, OT, negative-net cases); add a configurable API base URL for production builds.

✓ What is CORRECT (verified — no action needed)

To be fair to the original developer, these parts were checked and are done properly:

- **Income tax engine** — progressive bracket method (`salary x rate - deduction`) is the correct Ethiopian-style approach, and brackets live in the database, not in code.
- **Pension rates** — 7% employee / 11% employer come from the configurable `PensionRule` table with effective dates. 👍
- **Formula versioning** — payroll rules are versioned (`PayrollFormulaVersion` , `SalaryStructure` with `effectiveFrom`), so changing rules never rewrites historical payslips. This is how real payroll systems do it.
- **Maker-checker workflow** — prepare → check → approve → pay, with return-for-correction; every step recorded in `PayrollApproval` + the audit log.
- **Guard registration** — properly transactional (employee + profile + user account created atomically when MongoDB supports it, with compensating cleanup otherwise).
- **Rotation fairness math** — cycle-length and duty-percentage calculations are correct.
- **TypeScript** — backend and frontend both typecheck with 0 errors.

🔧 Recommended fix order (if money starts flowing soon)

1. **Install MongoDB** (Issue 17) — 30 minutes, unblocks everything.
2. **Issue 1** — fix the journal entry balance (add the employer-pension credit). Small change, stops silent accounting loss.
3. **Issue 2** — fix the guard pension base to actual worked salary. Requires ONE business decision (pension on worked basic vs gross), then a 3-line change.
4. **Issues 3 + 7** — fix staff OT ordering and remove/clean the manual OT override. Same code area, do together.
5. **Issue 4** — wire `assignment.hourlyRate` into rate resolution (one business decision on priority).
6. **Issue 5** — split the period status (add the two lock flags).
7. **Issue 12** — JWT secret fail-fast (2 lines).
8. **Issues 6, 8, 9, 10** — holiday-OT rate, loan balances, unassigned guards, month-end boundary.
9. **Issues 13–16** — uploads auth, password policy, CORS, rate limiting.
10. **Issues 11, 18, 19** — confirm the 48h rule with the business; hygiene and tests as time allows.

Vital-ERP Issues Report — compiled by Cline (AI coding agent) on 2026-09-14 from a full source-code review.

Scope: payroll engine, journal/accounting, attendance, rotations, workflow services, auth/security, environment — read line by line.

Companion document: "Vital-ERP-Documentation" (system overview + step-by-step user guide).